Міністерство освіти і науки України

Фаховий коледж зварювання та електроніки імені Є.О. Патона

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №5

навчальної дисципліни

ОПЕРАЦІЙНІ СИСТЕМИ

ФКЗЕ. F2ОС09. 05ЛР

Шифр роботи

Студент Забора К.

Група ПЗ-25-1/9

Перевірив Костенко А.О.

2026

Тема: Написання сценаріїв Windows Script Host.

Мета: ознайомитися з підсистемою Windows Script Host, навчитися писати сценарії мовами VBScript і JScript, працювати з файловою системою через об’єктну модель FileSystemObject, керувати оточенням і ярликами через WScript.Shell та отримувати відомості про систему через WMI.

Обладнання: ПК під керуванням ОС Windows 10 або Windows 11; Windows Script Host; текстовий редактор. Права адміністратора не потрібні.

Варіант: 9.

Хід роботи

1 Створив на робочому столі каталог LAB5 з підкаталогами src і out, наповнив src щонайменше п’ятьма файлами.

```
@echo off
set "L=%~dp0"
if not exist "%L%src" mkdir "%L%src"
if not exist "%L%out" mkdir "%L%out"
echo First test file> "%L%src\a_notes.txt"
echo id;name;score> "%L%src\b_data.csv"
echo 1;Ivan;90>> "%L%src\b_data.csv"
dir "%windir%" > "%L%src\c_dir.txt"
set > "%L%src\d_env.txt"
ver > "%L%src\e_ver.txt"
```

Рисунок 1 - Вміст .bat файлу для створення каталогу LAB5

2 Написав сценарій hello.vbs, який приймає аргумент, виводить привітання, ім’я сценарію та повне ім’я хоста. Запустив його двома способами - через wscript і через cscript - cscript виводить текст у консоль, його можна перенаправити (> file.txt), і він повертає код у %errorlevel%. wscript показує діалогове вікно з кнопкою OK і чекає натискання. У рядку «Хост» видно ….exe або ….exe.

```
Option Explicit
Dim sName, net
If WScript.Arguments.Count > 0 Then
    sName = WScript.Arguments(0)
Else
    sName = "незнайомцю"
End If
Set net = CreateObject("WScript.Network")
WScript.Echo "Привіт, " & sName & "!" & vbCrLf & _
    "Сценарій: " & WScript.ScriptName & vbCrLf & _
    "Хост (повне ім'я): " & WScript.FullName & vbCrLf & _
    "Комп'ютер: " & net.ComputerName
```

Рисунок 2 - Вміст hello.vbs

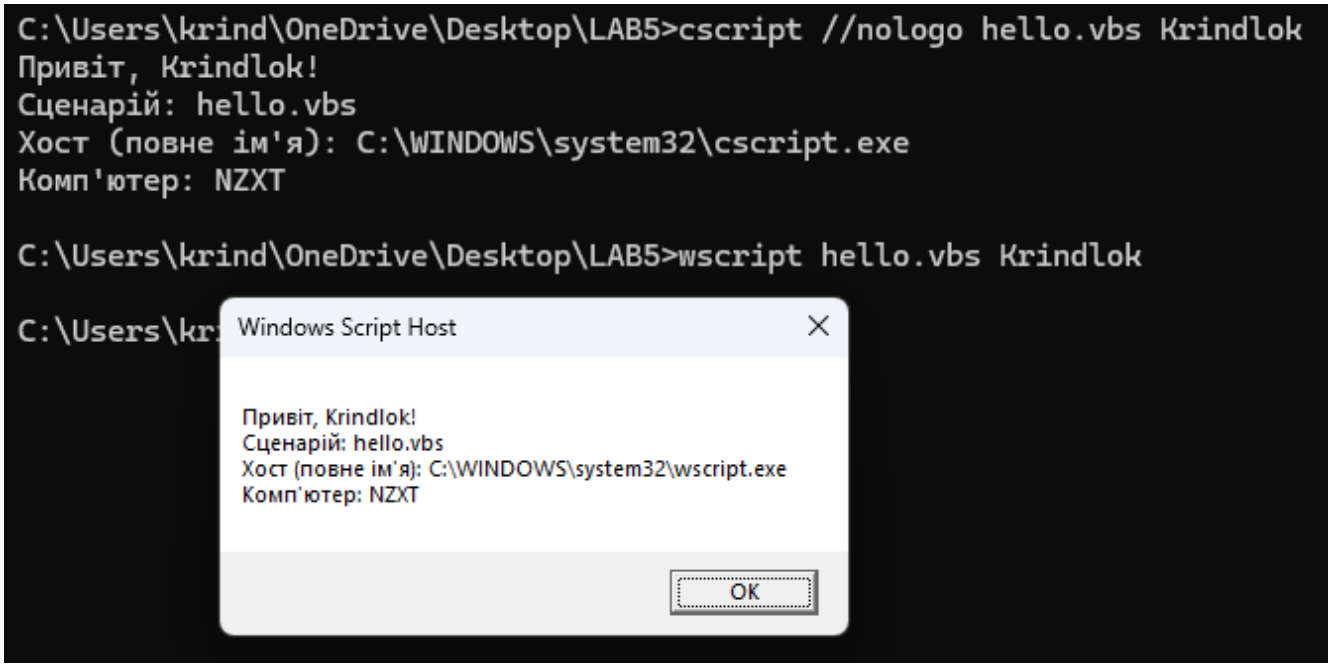


Рисунок 3 - Вивід двома способами

3 Написав сценарій інвентаризації каталогу src: перелік файлів з розмірами й датами, підсумковий рядок, запис результату у файл out.txt в кодуванні Unicode.

```
Option Explicit
Dim fso, base, srcDir, outDir, outPath, folder, f, ts, total, cnt
Set fso = CreateObject("Scripting.FileSystemObject")
base    = fso.GetParentFolderName(WScript.ScriptFullName)
srcDir  = base & "\src"
outDir  = base & "\out"
outPath = outDir & "\out.txt"

If Not fso.FolderExists(srcDir) Then
    WScript.Echo "Помилка: каталог не знайдено: " & srcDir
    WScript.Quit 2
End If

On Error Resume Next
If Not fso.FolderExists(outDir) Then fso.CreateFolder outDir
Set ts = fso.CreateTextFile(outPath, True, True)   ' True, True = перезапис, Unicode
If Err.Number <> 0 Then
    WScript.Echo "Помилка запису " & outPath & ": " & Err.Description
    WScript.Quit 4
End If
On Error GoTo 0

Set folder = fso.GetFolder(srcDir)
ts.WriteLine "Інвентаризація: " & srcDir
ts.WriteLine "Ім'я" & vbTab & "Розмір, байт" & vbTab & "Створено" & vbTab & "Змінено"
total = 0 : cnt = 0
For Each f In folder.Files
    ts.WriteLine f.Name & vbTab & f.Size & vbTab & f.DateCreated & vbTab & f.DateLastModified
    total = total + f.Size
    cnt = cnt + 1
Next
ts.WriteLine "Разом: файлів " & cnt & ", " & total & " байт"
ts.Close
WScript.Echo "Готово: файлів " & cnt & ", " & total & " байт"
```

Рисунок 4 - Сценарій інвентаризації каталогу src, з обробкою помилок

4 Реалізував основне завдання свого варіанта згідно з таблицею, використавши об’єкт, зазначений у варіанті.

```
Option Explicit
Dim sh, fso, cmd, workDir, rc, t0, verdict
Set sh  = CreateObject("WScript.Shell")
Set fso = CreateObject("Scripting.FileSystemObject")

If WScript.Arguments.Count > 0 Then
    cmd = WScript.Arguments(0)
Else
    cmd = "notepad.exe"
End If

If WScript.Arguments.Count > 1 Then
    workDir = WScript.Arguments(1)
    If Not fso.FolderExists(workDir) Then
        WScript.Echo "Помилка: каталог не знайдено: " & workDir
        WScript.Quit 2
    End If
    sh.CurrentDirectory = workDir
End If

WScript.Echo "Запуск: " & cmd
t0 = Timer
On Error Resume Next
rc = sh.Run(cmd, 1, True)          ' True = чекати завершення
If Err.Number <> 0 Then
    WScript.Echo "Помилка запуску: " & Err.Description & " (0x" & Hex(Err.Number) & ")"
    WScript.Quit 3
End If
On Error GoTo 0

Select Case rc
    Case 0:    verdict = "успішне завершення"
    Case 1:    verdict = "загальна помилка"
    Case 5:    verdict = "відмовлено у доступі"
    Case Else: verdict = "нестандартний код"
End Select

WScript.Echo "Тривалість: " & FormatNumber(Timer - t0, 2) & " с"
WScript.Echo "Код виходу: " & rc & " - " & verdict
WScript.Quit rc
```

Рисунок 5 - WScript.Shell, запуск з очікуванням і аналізом коду виходу

5 Додав до сценарію обробку помилки: звернення до неіснуючого файлу або каталогу дає зрозуміле повідомлення й ненульовий код повернення. Перевірив код через echo%errorlevel%.

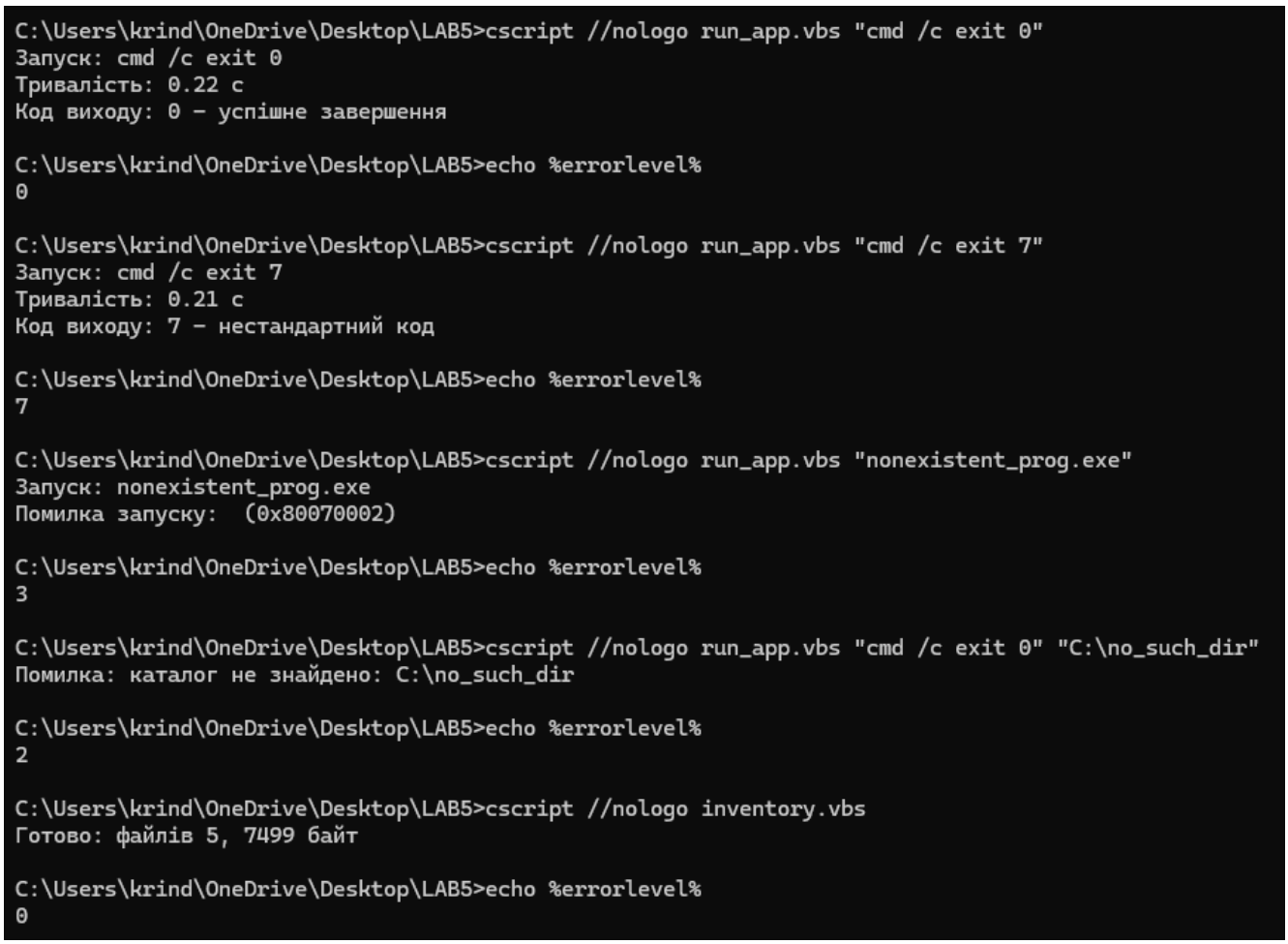


Рисунок 6 - Перевірка коду через echo%errorlevel%

6 Переписав один із написаних сценаріїв мовою JScript і порівняв обидві реалізації за обсягом та читабельністю таблицею. JScript трохи коротший, а try/catch читається краще, бо помилка обмежена блоком. On Error Resume Next діє на всі наступні рядки, поки його не скасують. VBScript має зручніші вбудовані функції форматування (FormatNumber, Hex).

```
var sh  = new ActiveXObject("WScript.Shell");
var fso = new ActiveXObject("Scripting.FileSystemObject");
var args = WScript.Arguments;
var cmd = args.length > 0 ? args(0) : "notepad.exe";

if (args.length > 1) {
    var workDir = args(1);
    if (!fso.FolderExists(workDir)) {
        WScript.Echo("Помилка: каталог не знайдено: " + workDir);
        WScript.Quit(2);
    }
    sh.CurrentDirectory = workDir;
}

WScript.Echo("Запуск: " + cmd);
var t0 = new Date().getTime();
var rc;
try {
    rc = sh.Run(cmd, 1, true);
} catch (e) {
    WScript.Echo("Помилка запуску: " + e.message + " (0x" + (e.number >>> 0).toString(16).toUpperCase() + ")");
    WScript.Quit(3);
}

var verdict;
switch (rc) {
    case 0:  verdict = "успішне завершення"; break;
    case 1:  verdict = "загальна помилка"; break;
    case 5:  verdict = "відмовлено у доступі"; break;
    default: verdict = "нестандартний код";
}

WScript.Echo("Тривалість: " + ((new Date().getTime() - t0) / 1000).toFixed(2) + " с");
WScript.Echo("Код виходу: " + rc + " - " + verdict);
WScript.Quit(rc);
```

Рисунок 7 - Переписаний сценарій мовою JScript

Таблиця 1 - Порівняння реалізацій за обсягом та читабельністю

	VBScript	JScript
Рядків коду (без коментарів і порожніх)	39	31
Символів коду	1151	1055
Обробка помилок	On Error Resume Next + Err.Number	try / catch

7 Виконав додаткове завдання свого варіанта. Якщо в Exec програма видає багато виводу, а сценарій його не читає, буфер переповниться і процес зависне. Тому вивід краще читати в циклі, поки процес працює.

```
Option Explicit
Dim sh, rc, ex, t0
Set sh = CreateObject("WScript.Shell")

WScript.Echo "=== Run з очікуванням ==="
t0 = Timer
rc = sh.Run("cmd /c echo Вивід Run іде у власне вікно & ping -n 3 127.0.0.1 >nul & exit 4", 1, True)
WScript.Echo "Код виходу: " & rc & ", пройшло " & FormatNumber(Timer - t0, 1) & " с"

WScript.Echo "=== Run без очікування ==="
t0 = Timer
rc = sh.Run("cmd /c ping -n 3 127.0.0.1 >nul & exit 4", 0, False)
WScript.Echo "Run повернув " & rc & " (завжди 0), пройшло " & FormatNumber(Timer - t0, 1) & " с"

WScript.Echo "=== Exec ==="
Set ex = sh.Exec("cmd /c echo Вивід Exec перехоплено сценарієм & exit 4")
Do While ex.Status = 0
    WScript.Sleep 100
Loop
WScript.Echo "StdOut: " & Trim(ex.StdOut.ReadAll)
WScript.Echo "ExitCode: " & ex.ExitCode & ", PID: " & ex.ProcessID
```

Рисунок 8 - Зміст порівняння

```
C:\Users\krind\OneDrive\Desktop\LAB5>cscript //nologo run_vs_exec.vbs
=== Run з очікуванням ===
Код виходу: 4, пройшло 2.3 с
=== Run без очікування ===
Run повернув 0 (завжди 0), пройшло 0.0 с
=== Exec ===
StdOut: ????? Exec ??????????? ?????????

ExitCode: 4, PID: 3472

C:\Users\krind\OneDrive\Desktop\LAB5>chcp 1251
Active code page: 1251
```

Рисунок 9 - Вивід порівняння

Таблиця 2 - Різниця між Run та Exec

	Run	Exec
Повертає	число (код виходу, якщо True; інакше 0)	об’єкт WshScriptExec
Очікування	параметр bWaitOnReturn	не чекає, треба опитувати Status (0 — працює, 1 — завершено)
Вивід	сценарій його не бачить	StdOut, StdErr, StdIn

	Run	Exec
програми		
Вікно	параметр intWindowStyle	немає такого параметра
PID і зупинка	немає	ProcessID, Terminate

Відповіді на контрольні питання

Високий рівень (творчий)

1. Організація має сотню робочих станцій і потребує щотижневого звіту про вільне місце на дисках. Опишіть, як розв’язати задачу засобами WSH: які об’єкти, як зібрати результати, які обмеження виникнуть.

Об’єкти: - WMI: клас Win32_LogicalDisk (DriveType = 3, поля Size, FreeSpace), підключення через GetObject("winmgmts:\\\\ім'я_ПК\\root\\cimv2"). - FileSystemObject для локальних дисків, Scripting.Dictionary або текстовий файл зі списком станцій.

Збирання результатів: - Сценарій читає список імен комп’ютерів і в циклі опитує кожен. - Для кожного диска обчислюється відсоток вільного місця. - Рядок станція; диск; розмір; вільно; % дописується в CSV/TXT на мережевій папці. - Запуск за розкладом (schtasks) або через GPO.

Обмеження: - Потрібні права адміністратора на віддалених станціях, відкриті DCOM/RPC у брандмауері. - Вимкнені або недоступні ПК дають помилку та затримку, тому потрібні On Error і запис невдач у журнал. - WSH не вміє паралельність, тож опитування сотні станцій по черзі повільне. - Мережеві диски можуть бути недоступні.

2. Сценарій .vbs — класичний носій шкідливого коду. Поясніть, які саме можливості WSH роблять його небезпечним і якими засобами системи цей ризик обмежують.

Чому небезпечний: - Сценарій запускається подвійним кліком без компіляції та пісочниці, з правами користувача. - Через COM-об’єкти він має широкі можливості: FileSystemObject (читання, запис, видалення файлів),

WScript.Shell (запуск програм, реєстр, змінні середовища), WMI (процеси, служби), мережеві об’єкти (завантаження та запуск коду). - Текст легко обфускувати, а розповсюджується він як вкладення пошти.

Чим ризик обмежується: - Права користувача й UAC (без прав адміністратора системні зміни недоступні). - Mark-of-the-Web і SmartScreen для завантажених файлів. - Антивірус та AMSI, який перевіряє скрипти під час виконання. - Політики: AppLocker/WDAC/SRP забороняють .vbs, WSH вимикається в реєстрі (HKLM\Software\Microsoft\Windows Script Host\Settings, Enabled = 0), у пошті блокуються такі вкладення, також можливий підпис скриптів.

3. Порівняйте WSH і PowerShell за трьома критеріями: доступ до системи, читабельність коду, наявність у сучасних версіях Windows. У яких випадках вибір WSH сьогодні виправданий?

Критерій	WSH	PowerShell
Доступ до системи	COM-об’єкти, WMI, FSO, Shell; без .NET	те саме + .NET, командлети, провайдер реєстру, віддалене керування (WinRM)
Читабельність	багатослівний синтаксис, слабка обробка помилок (On Error)	командлети Дієслово-Іменник, об’єкти в конвеєрі, try/catch, коротші сценарії
Наявність у Windows	є, але технологія застаріла: VBScript оголошено застарілим, у новіших Windows 11 це додатковий компонент, який планують вилучити	вбудований (5.1), активно розвивається

Коли вибір WSH виправданий сьогодні: - Підтримка старих сценаріїв (logon-скрипти, автоматизація Office/COM), які не варто переписувати. - Старі системи без PowerShell. - Середовища, де PowerShell заборонений політикою, а WSH дозволений. - Короткі завдання з COM-об’єктами.

Для нових розробок кращий PowerShell.

4. Той самий алгоритм реалізовано пакетним файлом і сценарієм WSH. Сформулюйте критерії, за якими ви оберете один із варіантів, і проілюструйте кожен критерій прикладом із цієї роботи.

Критерії вибору:

1. **Складність логіки.** Прості послідовності команд краще писати в .bat, наприклад setup.bat (створення src, out і п’яти файлів займає кілька рядків). Цикли за об’єктами та умови зручніші у WSH.
2. **Доступ до об’єктів системи.** Для властивостей файлів, процесів, дисків потрібні WSH і WMI: inventory.vbs бере розмір і дати через FileSystemObject, а в .bat для цього лише парсинг виводу dir.
3. **Обробка помилок і коди повернення.** run_app.vbs розрізняє коди (2 — немає каталогу, 3 — програму не запущено, інакше код програми); у .bat лише errorlevel і складніші перевірки.
4. **Робота з Unicode.** out.txt в UTF-16 пише CreateTextFile(..., True, True); у .bat кирилиця залежить від chcp і кодової сторінки консолі.
5. **Переносимість і залежності.** .bat працює скрізь без налаштувань, WSH потребує увімкненого хоста (cscript або wscript) і правильного кодування файлу (наприклад, UTF-8 у hello.vbs дало «кракозябри»).

Висновок: У роботі створено каталоги LAB5і LAB5та написано сценарії WSH: привітання з порівнянням хостів wscript і cscript, інвентаризацію каталогу із записом в Unicode-файл і запуск програми з очікуванням та аналізом коду виходу через WScript.Shell.Run (варіант 9). До сценаріїв додано обробку помилок: звернення до неіснуючого файлу чи каталогу дає зрозуміле повідомлення і ненульовий код, який перевіряється через echo %errorlevel% під cscript. Порівняння VBScript і JScript показало, що JScript коротший, а try/catch зручніший за On Error Resume Next. Run підходить для простого запуску з очікуванням і отриманням коду виходу, а Exec потрібен, коли треба перехопити вивід програми, дізнатися PID або зупинити процес.